Causal ModernTCN and Bi-LSTM for 6-DOF Ship Motion Time Series Prediction and Hyperparameter Optimization: A Code Analysis Report

1. Research Objective and Implementation Overview

The code analyzed in this report performs experiments that take past 6-DOF ship motion time series as input and predict future 6-DOF motions. The prediction targets are surge, sway, heave, roll, pitch, and yaw; the six variables are supplied jointly as input and predicted jointly as output, which constitutes a multivariate, multi-step forecasting problem.

The experiments were conducted in two stages. In the first stage, Optuna was used to explore the model architecture and training configuration. In the second stage, a model was re-initialized with the identified optimal configuration and retrained, and test performance was evaluated using the weights that yielded the lowest validation loss.

2. Prediction Problem and Main Experimental Settings

2.1. Definition of Inputs and Outputs

Let L_{in} denote the input sequence length, L_{out} the output sequence length, and D the number of degrees of freedom. The model input and output are then defined as follows:

$$X \in R^{B\times L_{in}\times D}, \qquad \hat{Y} \in R^{B\times L_{out}\times D}$$

where B denotes the batch size. In the present experiments, $L_{in} = 128$, $L_{out} = 128$, and $D = 6$. Because the sampling frequency of the data is 2 Hz, the interval between consecutive samples is 0.5 s; the input and output sequences therefore each consist of 128 samples, corresponding to 64 s. The fixed settings used throughout the experiments are summarized in Table 1, and the variable configuration is listed in Table 2.

2.2. Fixed Settings

Table 1. Fixed experimental settings.

Item	Value	Description
Data path	csv_dataset	Relative path from which the CSV files are read
Sampling frequency	2 Hz	Sampling frequency of the data
Input length	128 samples	Past 64 s
Output length	128 samples	Future 64 s
Window stride	16 samples	8 s interval
Number of input/output variables	6 each	6-IN → 6-OUT
Validation/test ratio	15% each	Rounded by the number of files
Training ratio	Remaining, approximately 70%	Files not assigned to validation or test
Default seed	42	Set for Python, NumPy, and PyTorch
Execution device	CUDA if available, otherwise CPU	CUDA device: cuda:0
Optimization objective	Minimization of validation MSE	Computed on normalized values

2.3. Variable Configuration

Table 2. Variable configuration: original column names, motion components, and display units.

Original column name	Motion component	Unit used for visualization and evaluation
x1	Surge	m
x2	Sway	m

x3	Heave	m
x4	Roll	deg
x5	Pitch	deg
x6	Yaw	deg

3. Data Loading and Preprocessing

3.1. CSV File Selection and Validation

The function load_id_data() reads the CSV files located directly under the data folder, sorted by file name. Files whose names begin with Wave_ are excluded; consequently, the wave files themselves are not included in the model input. Sea-state or operating-condition information contained in the file names is not extracted or used as input.

3.2. Condition-Level Data Splitting

The code treats each valid file as a single condition. The file indices are randomly shuffled and then assigned to the test, validation, and training sets, in that order. Because windows are generated within each set after the split, overlapping windows extracted from the same file are never assigned to both the training and test sets. This prevents the temporal leakage that can occur when windows are generated first and then randomly partitioned.

3.3. z-Score Normalization Based on Training Statistics

The per-DOF mean and standard deviation are computed from an array formed by concatenating all rows of the training files. These statistics are estimated from the training data only and are applied identically to the validation and test sets, which prevents distributional information from the validation and test sets from entering the scaler estimation. In addition, this normalization allows the translational and rotational components to be learned jointly in a normalized space despite their different numerical scales. The scaler and the file lists of each set are stored in scaler.json.

3.4. Sliding Window Generation

Instead of storing copies of the actual data, make_window_meta() stores (file index, start row index) pairs. The start index advances in steps of 16 rows, i.e., 0, 16, 32, and so on.

3.5. Dataset and DataLoader

TimeWindowDataset slices only the requested window from the original array, normalizes it, and returns it as a tensor. Because the windows are not copied and stored in advance, the memory duplication caused by overlapping windows is reduced. However, the full arrays of the original files and the concatenated training array used to compute the normalization statistics remain in memory. The DataLoader settings for each set are summarized in Table 3.

Table 3. DataLoader settings for the training, validation, and test sets.

Set	Shuffle	Last incomplete batch	Purpose
Training	Applied	Dropped (drop_last=True)	Weight update
Validation	Not applied	Retained	Selection of the optimal configuration and checkpoint
Test	Not applied	Retained	Final performance evaluation

All loaders use pin_memory=True, and the default number of workers is 4.

4. Model Architecture

4.1. Overall Tensor Transformations

Table 4. Tensor transformations in the Causal ModernTCN model.

Stage	Operation	Output shape
Input	Normalized past motions	(B, 128, 6)
Axis permutation	Transposition of the time and channel axes	(B, 6, 128)
Input projection	Causal convolution, 6 → C	(B, C, 128)
Feature extraction	N ModernTCN blocks	(B, C, 128)
Backbone output	Axis permutation	(B, 128, C)
Preparation for temporal projection	Axis permutation	(B, C, 128)
Temporal projection	Linear(128, 128), GELU, Dropout	(B, C, 128)
Preparation for output projection	Axis permutation	(B, 128, C)
Output projection	Linear(C, 6)	(B, 128, 6)

Here, C denotes the number of hidden channels and N the number of blocks. The complete sequence of tensor transformations is summarized in Table 4.

4.2. Causal Convolution

For a kernel size of k, CausalConv1d prepends $k - 1$ zeros to the left of the input and applies no padding on the right. The convolution itself has no padding, and the stride and dilation are left at their default value of 1. Consequently, the feature at output timestep t depends only on input timestep t and earlier timesteps, and the temporal length of the output is identical to that of the input. However, the earliest timesteps of the input window are affected by zero padding rather than by actual past observations. Causality is enforced in the convolutional feature extraction stage, whereas the final prediction head uses the entire observed input window. Because only past information available at the forecast origin is used, this does not constitute leakage of future ground truth.

4.3. Input Projection Layer

The input projection maps the six motion components to C hidden channels. Because it is an ordinary (non-grouped) convolution, it extracts temporal information while simultaneously combining information across the input degrees of freedom. Accordingly, the channels processed by the subsequent depthwise convolutions are learned hidden representations rather than the original channels that correspond one-to-one to individual degrees of freedom.

4.4. ModernTCN Block

Each block performs the following operations.
First, a depthwise causal convolution configured with groups=channels processes the temporal pattern of each hidden channel independently; no mixing across channels occurs at this stage.
Second, ChannelLayerNorm transposes the array to (B, T, C) and normalizes along the channel axis at each timestep of each sample. Because the statistics are computed neither over the entire time axis nor across batch samples, this operation does not introduce information from later timesteps into the current timestep.
Third, a pointwise feed-forward network implemented with kernel-size-1 convolutions expands the number of channels from C to $H = C \cdot expansion$. GELU and Dropout are applied, the channels are reduced from H back to C, and Dropout is applied again. This component mixes channel information within the same timestep.
Finally, a residual connection adds the block input to the transformed output.

The backbone output is passed to a linear layer along the time axis. The same temporal transformation matrix is applied to every hidden channel, mapping the 128 input timesteps to 128 output timesteps. GELU and Dropout are then applied, and the channel representation at each output timestep is mapped to the six degrees of freedom. All 128 future timesteps are predicted in a single forward pass. The overall architecture is illustrated in Figure 1.

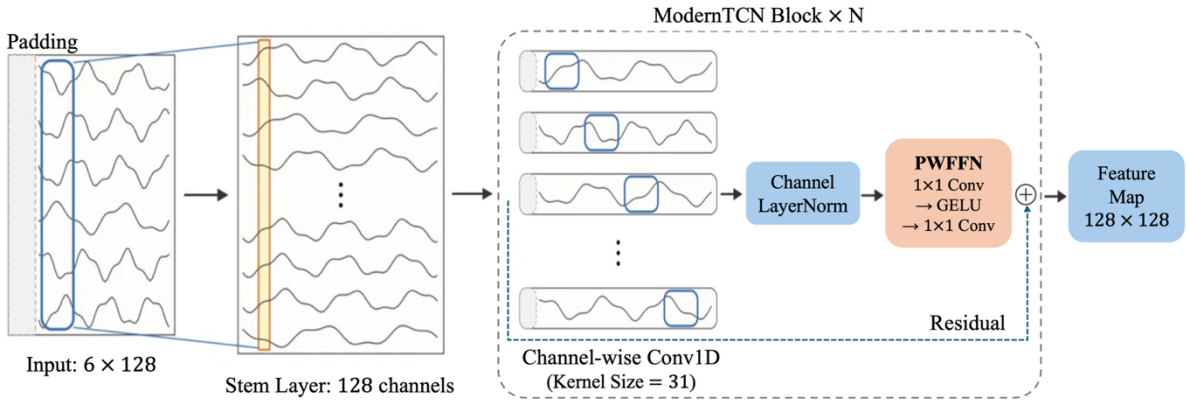


Figure 1. Architecture of the Causal ModernTCN model.

4.5. Bi-LSTM Single-Shot Model Architecture

To compare the predictive characteristics of a convolution-based model with those of a recurrent model, a Bi-LSTM single-shot model was employed. The observed past time series is encoded by a bidirectional LSTM, and the final forward and backward hidden states of the last layer are concatenated to predict the entire future window at once.

The forward LSTM processes the input window in chronological order, while the backward LSTM processes the same window in reverse. Because the backward pass also operates only within the 128 past samples already observed at the forecast origin, no future ground truth is included in the model input. However, unlike the Causal ModernTCN, which uses only past information at each timestep within the input window, the Bi-LSTM summarizes the observed window by processing it in both directions.

Let H denote the hidden dimension and L the number of LSTM layers; the final hidden state then has the shape (2L, B, H). The final forward and backward hidden vectors of the last layer are concatenated to form a representation of size (B, 2H). This concatenated representation is passed through a single linear layer and transformed into 6-DOF predictions for the 128 future timesteps. The corresponding tensor transformations are summarized in Table 5.

Table 5. Tensor transformations in the Bi-LSTM single-shot model.

Stage	Operation	Output shape
Input	Normalized past 6-DOF time series	(B, 128, 6)
Feature extraction	Multi-layer bidirectional LSTM	(2L, B, H)
State concatenation	Concatenation of the final forward and backward states of the last layer	(B, 2H)
Output projection	Linear(2H, 768)	(B, 768)
Output reshaping	Reshaping into the time and DOF axes	(B, 128, 6)

This architecture contains no separate recurrent decoder and performs no autoregressive computation in which predictions are fed back as inputs for the next timestep. The entire future window is therefore generated directly from a single hidden representation. In addition, no activation function or additional Dropout is applied to the output linear layer.

Whereas the Causal ModernTCN performs the temporal projection while retaining the timestep-wise features of the input window, the Bi-LSTM single-shot model compresses the input window into a fixed-dimensional final hidden representation before prediction. This difference reflects the distinct ways in which the two models preserve temporal information and construct the future window. The Bi-LSTM architecture is illustrated in Figure 2.

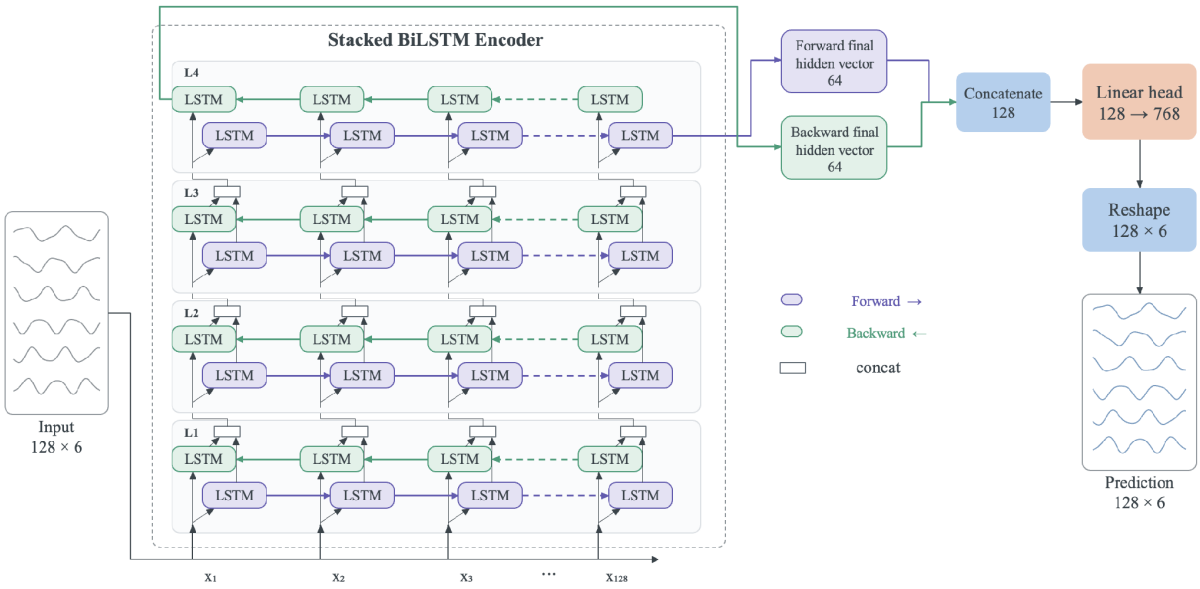


Figure 2. Architecture of the Bi-LSTM single-shot model.

5. Training Objective and Optimization

5.1. Training Procedure

The mean squared error (MSE) between the normalized predictions and the ground truth was used as the loss function.

The function train_epoch() sequentially performs training-mode setting, transfer of the inputs and targets to the device, gradient reset, prediction, MSE computation, backpropagation, gradient clipping, and the optimizer update. The maximum norm for gradient clipping is 1.0; rather than clipping each individual gradient to 1 or below, this rescales the gradients proportionally when the norm of the gradients over all parameters exceeds the threshold. The optimizer is AdamW, with the learning rate and weight decay selected by Optuna. The learning rate scheduler is ReduceLROnPlateau(mode="min", factor=0.5, patience=3), which halves the learning rate when the improvement in validation loss stagnates for a specified number of epochs.

5.2. Validation and Early Stopping

The function evaluate_epoch() computes the MSE and MAE using model.eval() and torch.no_grad(), without updating the weights. An epoch is regarded as an improvement when the validation loss falls below the previous minimum. Training is terminated early when no improvement is observed for 10 consecutive epochs. This criterion is applied in both the search stage and the final training stage, although the maximum number of epochs differs between the two (40 and 300, respectively). The training loss is computed while Dropout is active and the weights are being updated, whereas the validation loss is computed after the epoch with fixed weights and Dropout disabled.

6. Optuna Hyperparameter Search

6.1. ModernTCN Hyperparameter Search Space

Table 6. Hyperparameter search space for the ModernTCN model.

Parameter	Search range	Description
channels	64, 96, 128, 192, 256	Hidden representation dimension
num_blocks	Integers from 2 to 8	Network depth
kernel_size	7, 15, 23, 31, 39, 47, 55, 63	Temporal kernel size of the blocks
expansion	1.0, 2.0, 3.0, 4.0	Channel expansion ratio
dropout	0.00–0.30, step 0.05	Block Dropout
stem_kernel_size	3, 5, 7, 11, 15, 21	Input projection kernel size

head_dropout	0.00–0.30, step 0.05	Temporal projection head Dropout
learning_rate	0.0001–0.005, log scale	Learning rate
weight_decay	0.00001–0.01, log scale	AdamW regularization strength
batch_size	32, 64, 128	Training batch size

6.2. Procedure for Each Trial

Each trial resets the seed to 42, samples the hyperparameters, and creates a new model and data loaders. The model is trained for up to 40 epochs, and the validation MSE is recorded at every epoch. The objective value of a trial is the minimum validation MSE over the epochs performed rather than the loss at the final epoch. The number of model parameters, the number of completed epochs, and the lowest validation MSE to date are stored as trial attributes. The search space for the ModernTCN model is given in Table 6.

6.3. Sampler and Pruner

The sampler is TPESampler, with the number of startup trials set to 8. Subsequent trials are configured to proceed in the TPE manner, which exploits the accumulated evaluation results.

Pruning is handled by MedianPruner, configured with 8 startup trials and 8 warmup steps, where the epoch number is reported as the step in the code. Once the initial performance records have been accumulated and the warmup condition is satisfied, unpromising trials are terminated early on the basis of their intermediate results. Early stopping is based on loss stagnation within the same trial, whereas pruning utilizes intermediate evaluation information across the study. In the code, the pruning decision is made before the within-trial early-stopping decision.

6.4. Search Budget and Resumption

The number of trials was set to 40. If an existing SQLite study is found, the search is resumed from it, and additional trials are executed for the difference between 40 and the number of trials whose state is not RUNNING. This count includes trials in the PRUNED and FAIL states as well as those in the COMPLETE state.

Resumption via the SQLite database preserves the existing search results and performs additional trials. It does not store the model weights or optimizer state at intermediate epochs and therefore does not continue the training of an individual trial.

6.5. Bi-LSTM Hyperparameter Search Space

Table 7. Hyperparameter search space for the Bi-LSTM model.

Parameter	Search range	Description
hidden_dim	64, 128, 256, 512	Hidden state dimension
n_layers	Integers from 1 to 8	Number of LSTM layers
dropout	0.00–0.30, step 0.05	Dropout between LSTM layers
learning_rate	0.0001–0.005, log scale	Initial learning rate
weight_decay	0.00001–0.01, log scale	AdamW regularization strength
batch_size	32, 64, 128	Training batch size

7. Final Retraining with the Optimal Configuration

After the Optuna search is complete, if at least one trial in the COMPLETE state exists, the hyperparameters of the completed trial with the lowest validation MSE, study.best_params, are selected. The default seed of 42 is then reset, and a new model is constructed with the selected hyperparameters. This model is trained from scratch for up to 300 epochs; that is, the full retraining in the code trains the model with the optimal architecture again from random initialization. The weights are updated using the original training set, while early stopping and selection of the best weights are performed using the original validation set.

Whenever the validation MSE improves, the state_dict is saved to models/best_model.pt. Finally, the histories of the training MSE, validation MSE, validation MAE, and learning rate are stored in history.json.

8. Test Evaluation and Interpretation of Metrics

After the final retraining, the weights of the model with the lowest validation MSE are loaded and the test set is evaluated. The MSE and MAE are computed on the normalized data over all six degrees of freedom and the entire prediction window; these values carry no physical units. The prediction errors are then multiplied by the per-DOF standard deviations of the training data to restore the original units, and the RMSE is computed for each degree of freedom. The RMSE values for surge, sway, and heave are expressed in m, and those for roll, pitch, and yaw in deg.

9. Conclusions

9.1. Optimal ModernTCN Hyperparameters

Table 8. Optimal hyperparameters of the ModernTCN model (Trial 38).

Item	**Optimal value**
Trial number	38
Validation MSE	0.428539
Batch size	64
Channels	128
Number of blocks	4
Kernel size	55
Expansion ratio	2.0
Dropout	0.0
Head dropout	0.1
Stem kernel size	3
Learning rate	4.61×10^{-4}
Weight decay	2.61×10^{-5}

9.1.1. Comparison of the Top Trials

Table 9. Top ten ModernTCN trials ranked by validation MSE.

No.	Trial	Validation MSE	Channels	Blocks	Kernel	Expansion	Dropout
1	38	0.428539	128	4	55	2.0	0.00
2	21	0.430326	128	4	39	2.0	0.00
3	31	0.435885	128	3	47	3.0	0.00
4	32	0.437116	128	3	47	3.0	0.00
5	28	0.437514	128	3	47	3.0	0.00
6	33	0.439990	192	3	47	3.0	0.00
7	10	0.441250	128	8	63	3.0	0.30
8	11	0.442056	128	8	63	3.0	0.30
9	5	0.442903	128	4	39	2.0	0.05
10	12	0.443298	128	8	63	3.0	0.30

Table 8 lists the optimal hyperparameters identified by the search, and Table 9 compares the ten best trials. All five top-ranked trials used 128 hidden channels. The completed trials with 64 channels yielded validation MSE values of 0.49 or higher, and the models with 256 channels also recorded validation MSE values of at least 0.459. Within this search, 128 channels appeared to provide an appropriate balance between representational capacity and training stability.

The top five trials used three or four blocks, with kernel sizes of 39, 47, or 55. This indicates that relatively large convolutional kernels are advantageous for extracting long-range temporal patterns from the 2 Hz time series. The optimal trial, Trial 38, used a kernel size of 55 and four blocks. In addition, the backbone dropout of the top six trials was 0 in every case, suggesting that configurations with lower dropout rates are more likely to achieve higher performance in this setting.

The model of the optimal Trial 38 had 313,094 trainable parameters. In contrast, Trial 14 used approximately 3.31 million parameters but recorded a higher validation MSE of 0.459013 than the optimal model; that is, despite using roughly 10.6 times more parameters, its validation performance did not improve. Trial 21, whose performance was comparable to that of Trial 38, also used 314,118 parameters, and Trials 28, 31, and 32, each with approximately 330,000 parameters, were all ranked among the top trials. These results indicate that selecting

an appropriate temporal kernel size and number of blocks at a scale of roughly 310,000 to 340,000 parameters was more effective than indiscriminately increasing the model size. Sample predictions of the optimized ModernTCN are presented in Figure 3.

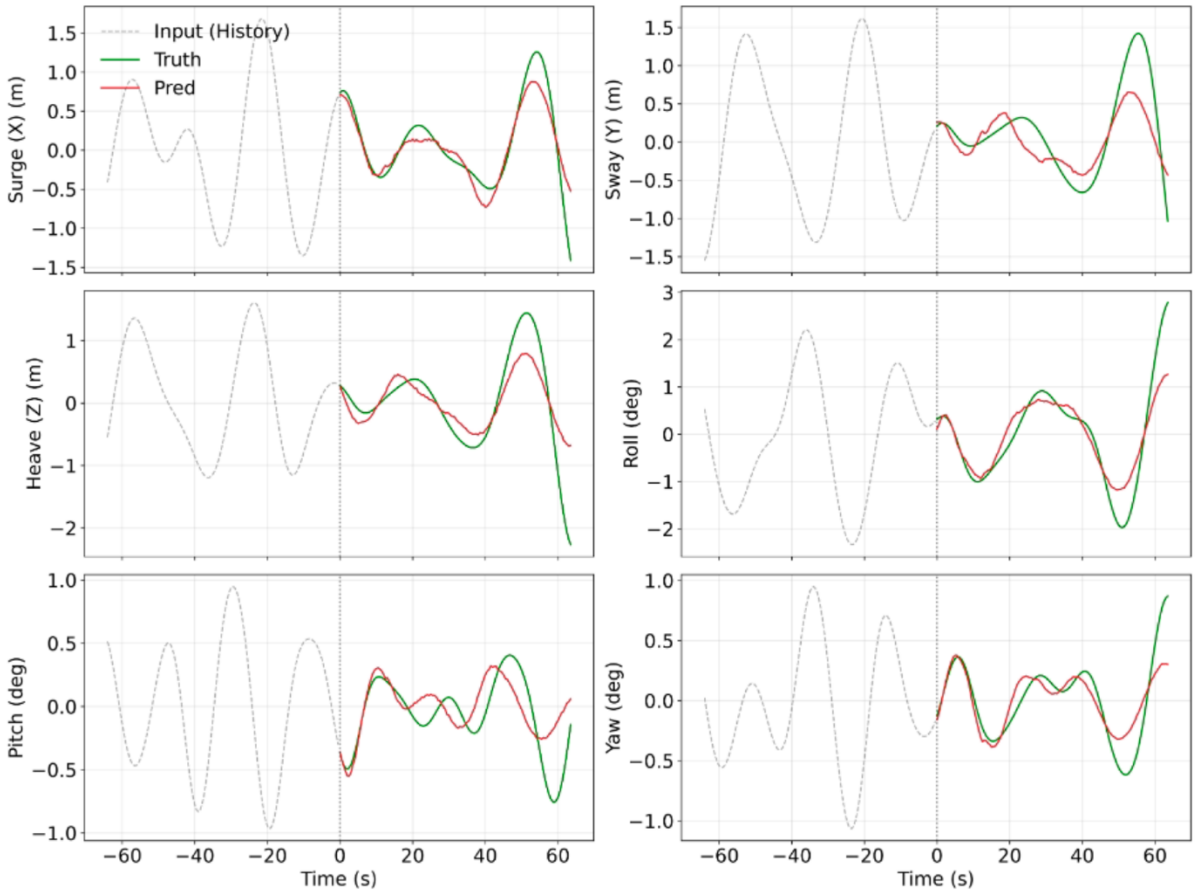


Figure 3. Sample predictions of the optimized ModernTCN.

9.2. Optimal Bi-LSTM Hyperparameters

Table 10. Optimal hyperparameters of the Bi-LSTM model (Trial 26).

Item	**Optimal value**
Trial number	26
Validation MSE	0.440436
Hidden dimension	64
Number of LSTM layers	4
Dropout	0.0
Learning rate	0.0013338544
Weight decay	0.0008422928
Batch size	128

9.2.1. Comparison of the Top Trials

Table 11. Top five Bi-LSTM trials ranked by validation MSE.

No.	Trial	Validation MSE	Hidden dimension	Layers	Dropout	Batch size	Number of parameters
1	26	0.440436	64	4	0.00	128	433,920
2	37	0.451676	512	5	0.00	64	28,115,712
3	19	0.460681	128	4	0.10	128	1,522,432
4	13	0.462594	128	6	0.20	128	2,312,960
5	38	0.462847	512	3	0.00	64	15,516,416

Table 10 lists the optimal Bi-LSTM hyperparameters, and Table 11 compares the five best trials. The optimal trial, Trial 26, used the fewest parameters among the top five configurations. The second-best trial, Trial 37, used approximately 64.8 times more parameters yet recorded a higher validation MSE. This indicates that, within the present search space, increasing the hidden dimension and model size alone did not improve prediction performance. Three of the top five configurations used no Dropout, and the optimal configuration likewise used a Dropout of 0. Consistent with the ModernTCN optimization results, this suggests that configurations with lower dropout rates are more likely to achieve higher performance. Sample predictions of the optimized Bi-LSTM are presented in Figure 4.

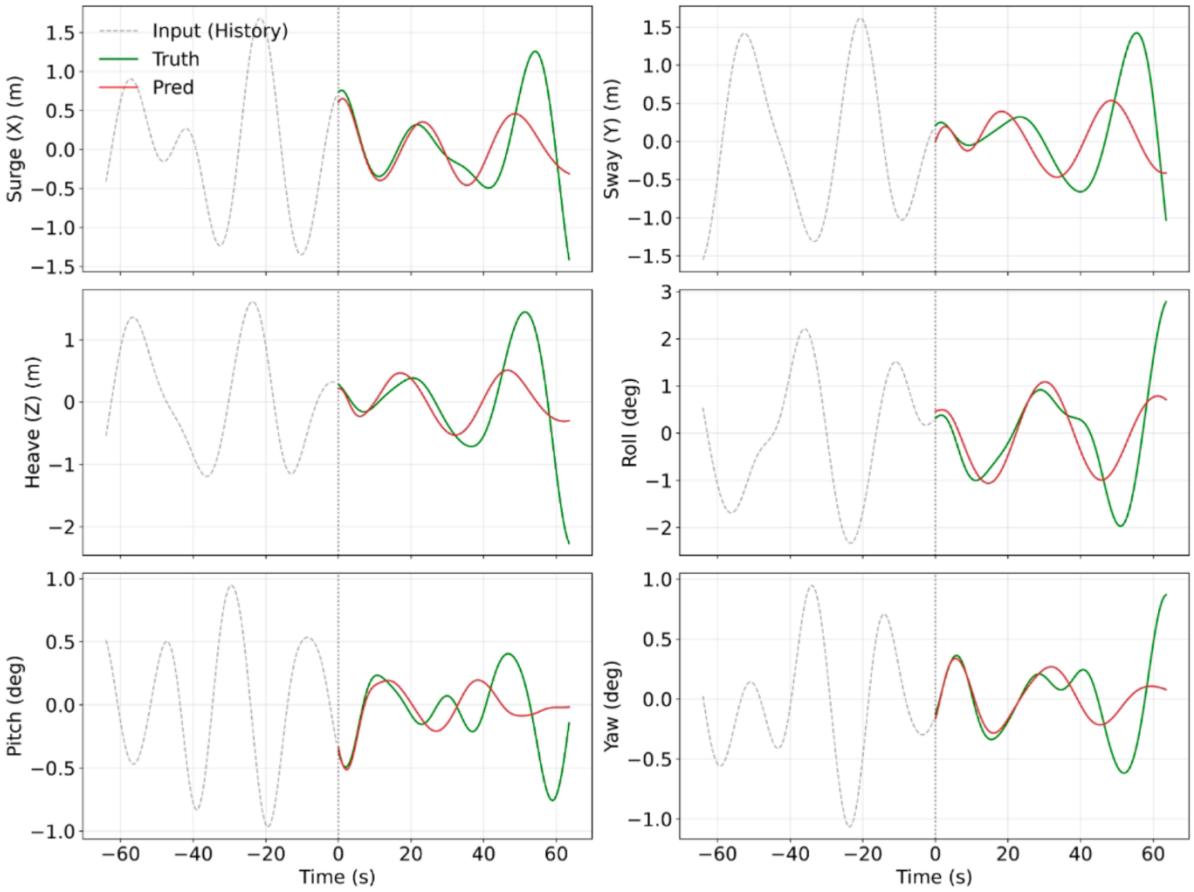


Figure 4. Sample predictions of the optimized Bi-LSTM.

Code Repository

- Bi-LSTM: https://github.com/yujin925/model_optimize
- modernTCN: https://github.com/yangjunahn/lrf-sswu/blob/main/py_modernTCN_std.py